QAgent AI

Protótipo de agente de IA para automação de testes E2E · Documentação

1. O que o sistema faz

O QAgent recebe, em linguagem natural, **o que você quer testar** em uma página web e a **URL** dela. A partir disso ele:

- abre a página e levanta quais campos, botões e links existem;
- pede a um modelo de IA (LLM) um **plano de teste**, isto é, uma lista de passos como preencher, clicar e verificar um texto;
- executa esse plano num navegador real (Chromium, via Playwright);
- se um passo falhar, devolve o erro ao modelo para que ele **corrija o plano**, repetindo até 3 vezes;
- quando o teste passa, grava um arquivo de teste Playwright (`.spec.ts`) e um relatório em JSON.

Em resumo: você descreve o objetivo, e o agente escreve, roda e ajusta o teste.

2. Fluxo de funcionamento

1. Perceber	2. Planejar	3. Agir	4. Observar	5. Corrigir
snapshot da página	LLM gera o plano em JSON	Playwright executa os passos	erro e screenshot se falhar	novo plano (até 3 tentativas)

Se a execução passa, o ciclo termina e o teste é exportado. Se as 3 tentativas falham, o relatório registra todas elas e o programa termina com código de saída 1.

3. Componentes

Arquivo	Responsabilidade
`src/agent.mjs`	Linha de comando e loop principal (gerar, executar, corrigir). Também grava o relatório e o `.spec.ts` . No modo demo, sobe um servidor local com o app de exemplo.
`src/snapshot.mjs`	Lê a página e devolve URL, título, títulos (h1 a h3) e a lista de elementos interativos (`input` , `button` , `a` , `select` , `textarea`) com um seletor sugerido para cada um.
`src/llm.mjs`	Camada de IA. Tem o provedor Claude (com saída estruturada), provedores compatíveis com a API da OpenAI (Gemini, Groq, Ollama, OpenAI) e um LLM simulado para testes offline.
`src/executor.mjs`	Executa os passos no Playwright e para no primeiro que falhar, tirando um screenshot.
`demo/app.html`	Aplicação de login de exemplo para experimentar o agente.
`reports/`	Saída: `ultimo-relatorio.json` e `falha-N.png` .
`generated/`	Saída: `agent.spec.ts` , o teste gerado.

4. O plano de teste

O modelo não escreve código livre. Ele devolve um JSON com um nome e uma lista de passos, o que facilita validar e executar com segurança.

Ação	O que faz
`goto`	Abre a URL em `value` e espera a rede acalmar.
`fill`	Preenche o campo `selector` com `value`.
`click`	Clica no elemento `selector`.
`expect_visible`	Espera o elemento `selector` ficar visível.
`expect_text`	Espera o texto `value` aparecer, dentro do `selector` ou, se não houver, na página inteira.
`expect_url`	Espera a URL conter `value`.

```
{
  "name": "Login com credenciais válidas",
  "steps": [
    { "action": "goto", "value": "http://localhost:3000/" },
    { "action": "fill", "selector": "#email", "value": "qa@teste.com" },
    { "action": "fill", "selector": "#password", "value": "123456" },
    { "action": "click", "selector": "[data-testid=\"submit-login\"]" },
    { "action": "expect_text", "value": "Login realizado com sucesso" }
  ]
}
```

Cada passo espera no máximo 3 segundos. As regras dadas ao modelo são: usar apenas seletores presentes no snapshot, começar com `goto` e terminar com ao menos uma verificação.

5. Como usar

Instalação

```
npm install
npx playwright install chromium
```

Demonstração offline (sem chave de API)

```
npm run demo
```

Usa o app de exemplo e o LLM simulado, que erra o seletor de propósito na 1ª tentativa para mostrar a auto-correção.

Em uma página sua

```
node src/agent.mjs --url https://seusite.com --goal "o que testar" --headed
```

Opção	Descrição
`--url`	Página a testar.
`--goal`	Objetivo do teste em linguagem natural.
`--headed`	Mostra o navegador durante a execução.
`--mock`	Usa o LLM simulado (só serve para o app de demonstração).
`--demo`	Sobe o app de exemplo e testa nele.

6. Provedores de IA

O provedor é escolhido pela variável `QAGENT_PROVIDER`. Sem ela, usa o Claude (`ANTHROPIC_API_KEY`).

Provedor	Configuração	Observação
anthropic (padrão)	`ANTHROPIC_API_KEY`	Modelo padrão `claude-sonnet-5-5`.
gemini	`QAGENT_PROVIDER=gemini` e `LLM_API_KEY`	Há plano gratuito.
groq	`QAGENT_PROVIDER=groq` e `LLM_API_KEY`	Há plano gratuito.
ollama	`QAGENT_PROVIDER=ollama`	Roda local, sem chave nem envio de dados para fora.
openai	`QAGENT_PROVIDER=openai` e `LLM_API_KEY`	Pago.

`QAGENT_MODEL` troca o modelo e `LLM_BASE_URL` troca o endereço da API.

7. O que o sistema produz

- **Console:** o resultado de cada tentativa, indicando o passo que falhou e o motivo.
- **reports/ultimo-relatorio.json:** objetivo, URL, resultado final e o histórico de todas as tentativas, com o plano e o erro de cada uma.
- **reports/falha-N.png:** screenshot do momento da falha na tentativa N.
- **generated/agent.spec.ts:** teste Playwright pronto, criado só quando o teste passa.

O `.spec.ts` gerado usa a URL da execução. No modo demo ela inclui uma porta aleatória, então é preciso trocá-la por um `baseURL` antes de reaproveitar o teste.

8. Limitações conhecidas

- Enxerga uma página por vez: não planeja fluxos que passam por várias telas e não trata iframes nem shadow DOM.
- Não tem login reutilizável, fixtures ou gestão de massa de dados.
- A correção usa apenas a mensagem de erro. O screenshot é gravado, mas ainda não é enviado ao modelo.
- As verificações são tão boas quanto o objetivo informado: um objetivo vago pode gerar um teste que passa sem validar nada importante.
- Modelos menores tendem a errar mais seletores e a precisar de mais tentativas.

9. Cuidados de segurança

- O objetivo e o snapshot da página são enviados ao provedor de IA escolhido (exceto com Ollama, que é local). Use apenas dados e credenciais de teste.
- Credenciais escritas no `--goal` ficam no histórico do terminal e no relatório JSON.
- Revise o `.spec.ts` gerado antes de incluí-lo numa suíte, principalmente as verificações.